

CUR T-SVDM: T-CURM

Joshua Huang

Luca Grossmann

Luke Ito

December 20, 2023

Abstract

Tensors have become a popular way of storing data, due to the ease with which they handle multidimensional information. For example: a video might be stored in a tensor with “height”, “width” and “greyscale” dimensions. Despite how common tensors have become in data storage, the tendency is still to “flatten” or “matricize” when it comes to analyzing or compressing the data itself. “Flattening” the tensor opens the door for powerful traditional linear algebraic tools such as principal component analysis (PCA). However, doing so ignores higher dimensional correlations that might be stored in the data. This higher dimensional structure could prove useful in finding optimal compression of the data. Recently there have been major advances in the field of low order tensor decomposition methods, with modern methods such as the T-SVDM, introduced in [4] leading to new optimality guarantees in compression. Despite achieving remarkable compression, these methods can run into time complexity issues as data sets continue to increase in size. In this paper we generalize the CUR decomposition [6], a randomized low rank matrix decomposition method to the $\star_M$ tensor algebraic framework. Generalizing the CUR decomposition to the multilinear case will let us use the method’s complexity improvements and guaranteed error bounds while still preserving intra-dimensional correlations.

1 Introduction

In our paper we will be linking the T-SVDM based on the $\star_m$ product introduced in [4] with the CUR decomposition introduced in [6]. A randomized version of the t-SVD (an earlier version of the T-SVDM which used the Discrete Fourier Transform as M [3]) was developed in [7]. They used the Randomized SVD, a projection based algorithm. We thought it would be interesting to explore the T-SVDM tensor using a random sampling approach. In this paper we provide background the $\star_M$ product, the Eckart-Young optimality of the T-SVDM and then the CUR decomposition. Then we introduce our algorithm and discuss the complexity and error bounds associated with it. We conclude our paper with a numerical example and further questions that we had in our exploration.

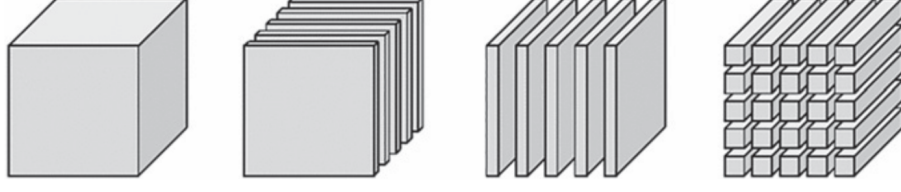


Fig. 1. Fibers and slices of $m \times p \times n$ tensor A . Left to right: Frontal slices, denoted either $A^{(j)}$ or $A_{:, :, i}$; lateral slices denoted $\tilde{A}_j$ or $A_{:, j, :}$; tube fibers $\mathbf{a}_{ij}$ or $A_{ij, :}$.

Figure 1: Image and caption from [4]

2 T-SVDM

2.1 Background and Notation

2.1.1 Tensor Basics

A tensor is a multidimensional array. As this paper is focused on 3-way tensors (3-dimensional array), the notation introduced will relate to the third order case.

Let $A \in \mathbb{C}^{m \times p \times n}$ be a 3-way tensor. Since the $\star_M$ product based algebra is orientation dependent, we will relate each dimension to the following values:

- $m := \text{samples}$
- $p := \text{variables}$
- $n := \text{time points}$

Attributing the following values, while inconsequential to the underlying mathematics, is useful to build an intuitive understanding of the algorithm.

We will be using the tensor Frobenius norm, defined as $\|A\|_F^2 = \sum_{i,j,k} |A_{i,j,k}|^2$, Where i denotes the row, j denotes the column, and k denotes the slice of the tensor. We will define the i th *frontal slice* of A as $A_{:, :, i}$. A *tube fiber* of A is $A_{i, j, :}$.

With this notation in place, we can introduce the tensor framework we will be using, which was outlined in [4].

2.1.2 Tensor Matrix Products

We will be using the mode 3 tensor matrix product introduced in [5].

We first need to define the mode 3 unfolding of a tensor, denoted $A_{(3)}$:

$$A_{(3)} := [sq(A_{:, 1, :})^T, sq(A_{:, 2, :})^T, \dots, sq(A_{:, p, :})^T] \in \mathbb{R}^{n \times mp}$$

Where the Sq operator turns a lateral slice $X \in \mathbb{C}^{m \times 1 \times n}$ into $\mathbf{X} \in \mathbb{C}^{m \times n}$ as follows:

$$\mathbf{X} = sq(X)$$

$A_{(3)} \in \mathbb{C}^{n \times mp}$ is the concatenation of the lateral slices of A or $(A_{:,i,:})$ into a matrix.

The mode 3 tensor matrix product is defined as follows:

Let $\mathbf{M}$ be an $r \times n$ matrix and $A \in \mathbb{C}^{m \times p \times n}$ be a 3-way tensor. The mode 3 product of A and $\mathbf{M}$, denoted $A \times_3 \mathbf{M}$, is:

$$A \times_3 \mathbf{M} := \mathbf{M}A_{(3)}$$

where $\mathbf{M}A_{(3)} \in \mathbb{R}^{r \times mp}$. We then reshape the resulting product into an $m \times p \times r$ tensor. Another way to interpret $A \times_3 \mathbf{M}$ is that for each *tube fiber* in $A := A_{i,j,:}$ we are computing the matrix vector product $\mathbf{M}A_{i,j,:}$. Then we can interpret the operation $A \times_3 \mathbf{M}$ as applying the linear operator $\mathbf{M}$ onto each sample for each variable. If $\mathbf{M}$ is an orthogonal matrix, which we will assume for the T-SVDM, we can interpret the operation as rotating each tube fiber in A into a transform domain, which is more interpretable.

2.1.3 The Family of $\star_M$ Tensor-Tensor Products

Let $\mathbf{M}$ be any invertible $n \times n$ matrix and $A \in \mathbb{C}^{m \times p \times n}$. We will use a hat to denote a tensors in the transform domain:

$$\hat{A} := A \times_3 \mathbf{M}$$

Note that because $\mathbf{M}$ is an $n \times n$ matrix, $\hat{A}$ will have the same dimensions as A . In addition to this, $\hat{A}$ must be interpreted within the context of the $\mathbf{M}$ being using to transform it, as different $\mathbf{M}$ will lead to very different $\hat{A}$. We can now define the $\star_M$ product introduced in [2]:

Algorithm ($\star_M$ Operation):

Input: $A \in \mathbb{C}^{m \times p \times n}$, $B \in \mathbb{C}^{p \times l \times n}$, invertible $\mathbf{M} \in \mathbb{C}^{n \times n}$.

1. $\hat{A} \leftarrow A \times_3 \mathbf{M}, \hat{B} \leftarrow B \times_3 \mathbf{M}$
2. **for** $i = 1, \dots, n$ **do**:
3. $\hat{C}_{:,i} = \hat{A}_{:,i} \hat{B}_{:,i}$
 end for
4. $C = \hat{C} \times_3 \mathbf{M}^{-1}$
5. **return** C

Now $C = A \star_M B$. In practice, this algorithm first puts tensors A and B into the transform domain. We then define each front slice of $\hat{C}$ as the product of the corresponding front slices of $\hat{A}$ and $\hat{B}$. Next, we take the mode 3 product of $\hat{C}$ with $\mathbf{M}^{-1}$ to return to the original domain.

2.1.4 Tensor Algebraic Framework

Definition (Conjugate Transpose):

Given $A \in \mathbb{C}^{m \times p \times n}$, its conjugate transpose $A^H \in \mathbb{C}^{p \times m \times n}$, under $\star_M$ is defined as follows:

$$(A^H)_{:,i} = (A_{:,i})^H$$

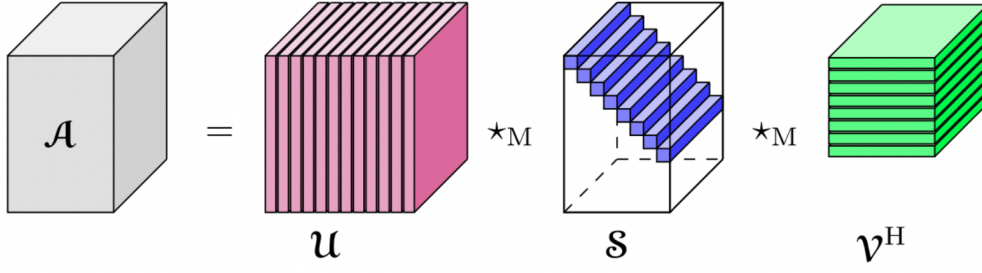


Figure 2: tSVDM (taken from [4])

In other words, the conjugate transpose of a matrix is equal to the conjugate transpose of each of its frontal slices stacked onto one another.

Definition (Identity Tensor):

The identity tensor $I \in \mathbb{R}^{m \times m \times n}$ satisfies the following:

$$A \star_M I = I \star_M A = A$$

Each frontal slice of $\hat{I} = I \times_3 M$ is an identity matrix.

Definition (Unitary Tensor):

$Q \in \mathbb{C}^{m \times m \times n}$ is called $\star_M$ unitary if:

$$Q^H \star_M Q = I = Q \star_M Q^H$$

2.1.5 Tensor $\star_M$ M-SVD's

While the $\star_M$ product is valid for any invertible $\mathbf{M}$, by enforcing that $\mathbf{M}$ must be a scalar multiple of a unitary matrix, we can use an extension of unitary invariance of the Frobenius norm to prove the Eckart-Young optimality of the T-SVDM.

Claim (Unitary Invariance under the Frobenius Norm):

Let $\mathbf{M} \in \mathbb{C}^{n \times n}$ be defined as $\mathbf{M} := c\mathbf{W}$ for unitary $\mathbf{W}$ and $c \neq 0$, and $\mathbf{Q} \in \mathbb{C}^{m \times m \times n}$ be a $\star_M$ -unitary tensor, then for $B \in \mathbb{R}^{m \times k \times n}$:

$$\|Q \star_M B\|_F = \|B\|_F$$

Proof: Suppose $\mathbf{M} = c\mathbf{W}$ where $\mathbf{W}$ is unitary. Then $\mathbf{M}^{-1} = \frac{1}{c}\mathbf{W}^H$. This implies the following:

$$\begin{aligned} \|\hat{B}\|_F &= \|B \times_3 \mathbf{M}\|_F = \|c\mathbf{W}B_{(3)}\|_F && \text{(by definition)} \\ &= c\|B_{(3)}\|_F && \text{(orth. invariance of } \|\cdot\|_F) \\ &= c\|B\|_F && \text{(folding preserves } \|\cdot\|_F) \end{aligned}$$

Let $C = Q \star_M B$. Using the previous result,

$$\begin{aligned}
\|B\|_F^2 &= \frac{1}{|c|^2} \|\hat{B}\|_F^2 && \text{(prev. result)} \\
&= \|B\|_F^2 && \text{(unitary inv. of } \|\cdot\|_F) \\
&= \frac{1}{|c|^2} \|\hat{B}\|_F^2 && \text{(tensor-tensor product)} \\
&= \frac{1}{|c|^2} \sum_{i=1}^p \|\hat{Q}_{:, :, i} \hat{B}_{:, :, i}\|_F^2 \\
&= \frac{1}{|c|^2} \|\hat{C}\|_F^2 \\
&= \|C\|_F^2 \\
&= \|Q \star_M B\|_F^2
\end{aligned}$$

$\|B\|_F^2 = \|Q \star_M B\|_F^2$, so the $\star_M$ operation does not change the Frobenius Norm of a matrix. $\square$

With this in mind, we define the Tensor SVD induced by a fixed $\star_M$ operator.

Definition (Tensor SVD):

Let $A \in \mathbb{R}^{m \times p \times n}$. The full $\star_M$ Tensor SVD (t-SVDM) of A is as follows:

$$A = U \star_M S \star_M V^H = \sum_{i=1}^r U_{:, i, :} \star_M S_{i, i, :} \star_M V_{:, i, :}^H$$

Where $U \in \mathbb{R}^{m \times m \times n}$, $V \in \mathbb{R}^{p \times p \times n}$ are $\star_M$ unitary tensors, $S \in \mathbb{R}^{m \times p \times n}$ is a tensor whose frontal slices are diagonal, and $r \leq \min(m, p)$ is the number of nonzero tubes in S. The algorithms for the Full t-SVDM, introduced in [2], takes the following steps:

Algorithm (t-SVD):

Input: $A \in \mathbb{C}^{m \times p \times n}$, invertible $\mathbf{M} \in \mathbb{C}^{n \times n}$

1. $\hat{A} \leftarrow A \times_3 \mathbf{M}$
2. **for** $i = 1, \dots, n$ **do**
3. $[\hat{U}_{:, :, i}, \hat{S}_{:, :, i}, \hat{V}_{:, :, i}] = \text{svd}(\hat{A}_{:, :, i})$
 end for
4. $U = \hat{U} \times_3 \mathbf{M}^{-1}, S = \hat{S} \times_3 \mathbf{M}^{-1}, V = \hat{V} \times_3 \mathbf{M}^{-1}$
5. **return** U, S, V

This algorithm brings A into the transform space by applying an orthogonal M, then computing and storing the SVD output of every sheet of $\hat{A}$. Finally, we return to the spacial domain by applying the inverse transformation. The primary motivation behind this algorithm is that with smart choices in orthogonal M, $\hat{A}$ can be much more compressible than A given the "rotation" of each of the tube fibers according to M. An example M that has been used with success in the past was the Discrete Fourier Transform matrix.

Now that we have the algorithms spelled out, we can work towards the major result

of [4], the proof of Eckart-Young-like optimality.

Corollary: Assume $\mathbf{M} = c\mathbf{W}$ with $c \neq 0$, and $\mathbf{W}$ unitary. Then given the t-SVDM of A over $\star_M$ as defined above:

$$\|A\|_F^2 = \|S\|_F^2 = \sum_{i=1}^{\min(p,m)} \|S_{i,i,:}\|_F^2$$

with $\|S_{1,1,:}\|_F^2 \geq \|S_{2,2,:}\|_F^2 \geq \dots$

Proof: The first equality is due to:

$$\|A\|_F^2 = \|U \star_M S \star_M V^H\|_F^2 = \|U \star_M S\|_F^2 = \|S\|_F^2$$

Which is given by the unitary invariance claim from above.

The second equality comes from the definition of the Frobenius norm and the fact that S only has non-zero entries on the diagonal.

The ordering property follows from:

$$\|S_{i,i,:}\|_F^2 = \frac{1}{c^2} \|\hat{S}_{i,i,:}\|_F^2 = \frac{1}{c^2} \sum_{j=1}^n (\hat{\sigma}_i^{(j)})^2$$

Where $(\hat{\sigma}_i^{(j)})$ is the i th largest singular value on frontal slice of $\hat{S}$. Since we ordered the singular values when we took the SVD of each sheet initially, the result follows. $\square$

Definition (t-rank):

The t-rank of a tensor A is the number of nonzero singular tubes $(S_{i,i,:})$ in the t-SVDM.

Theorem (Eckart-Young Theorem For Matrices):

Suppose $A, B \in \mathbb{R}^{m \times n}$ and $\text{rank}(B) = k \leq r = \text{rank}(A)$. Let A_k be the rank k approximation of A by the SVD:

$$A = USV^T \\ A_k = \sum_{i=1}^k u_i s_i v_i^T$$

Then we have that:

$$\|A - B\|_F^2 \geq \|A - A_k\|_F^2$$

In other words, A_k is the best rank k approximation to A under the Frobenius norm.

Theorem (Eckart-Young Theorem For Tensors):

The Eckart-Young Theorem For tensors is the primary result of [4], and the reason for which we are picking the T-SVDM for our algorithm. Truncating the t-product based t-SVDM of A will give us an optimal low t-rank approximation of A under the Frobenius norm. This result was shown for the t-product in [2] and extended to the $\star_M$ product in [4].

$$A_k = U_{:,1:k,:} \star M S_{1:k,1:k,:} \star_M V_{:,1:k,:}^T = \sum_{i=1}^k U_{:,i,:} \star_M S_{i,i,:} \star M V_{:,i,:}^T$$

Where $\mathbf{M} = c\mathbf{W}$. Then A_K is the best Frobenius norm approximation to A of the set $\Gamma = \{C = X \star_M Y | X \in \mathbb{R}^{m \times k \times n}, Y \in \mathbb{R}^{k \times p \times n}\}$, the set of all t-rank k tensors under $\star_M$ of the same dimensions as A . The squared error is $\|A - A_K\|_F^2 = \sum_{i=k+1}^r \|S_{i,:}\|_F^2$ where r is the t-rank of A .

Proof: First, we show that $\|A - A_K\|_F^2 = \sum_{i=k+1}^r \|S_{i,:}\|_F^2$. Rewriting A as:

$$A = \sum_{i=1}^r U_{:,i,:} \star_M S_{i,i,:} \star_M V_{:,i,:}^T, \text{ and } A_K = \sum_{i=1}^k U_{:,i,:} \star_M S_{i,i,:} \star_M V_{:,i,:}^T$$

The result arrives after computing the difference of the two summations.

Now, we show that A_K minimizes $\|A - A_K\|_F^2$ for all t-rank k tensors: Let $B = X \star_M Y$. Then we know that:

$$\|A - B\|_F^2 = \frac{1}{c^2} \|\hat{A} - \hat{B}\|_F^2 = \frac{1}{c^2} \sum_{i=1}^n \|\hat{A}_{:,i} - \hat{B}_{:,i}\|_F^2$$

The first equality comes from the following:

$$\|\hat{A} - \hat{B}\|_F^2 = \|(A \times_3 \mathbf{M}) - (B \times_3 \mathbf{M})\|_F^2 \quad (1)$$

$$= \|\mathbf{M}A_{(3)} - \mathbf{M}B_{(3)}\|_F^2 \quad (2)$$

$$= \|c\mathbf{W}A_{(3)} - c\mathbf{W}B_{(3)}\|_F^2 \quad (3)$$

$$= \|c\mathbf{W}(A_{(3)} - B_{(3)})\|_F^2 \quad (4)$$

$$= c^2 \|A_{(3)} - B_{(3)}\|_F^2 \quad (5)$$

$$= c^2 \|A - B\|_F^2 \quad (6)$$

The second equality, $\frac{1}{c} \|\hat{A} - \hat{B}\|_F^2 = \frac{1}{c} \sum_{i=1}^n \|\hat{A}_{:,i} - \hat{B}_{:,i}\|_F^2$, follows from the definition of the Frobenius norm.

By definition, $\hat{B}_{:,i} = \hat{X}_{:,i} \star_M \hat{Y}_{:,i}$ is a rank k matrix. We know that the best rank- k approximation of $\hat{A}_{:,i}$ is $U_{:,1:k,i} \star_M S_{1:k,1:k,i} \star_M V_{:,1:k,i}^T$ by the Eckart-Young Theorem for the Singular Value Decomposition. Thus:

$$\|\hat{A}_{:,i} - \hat{U}_{:,1:k,i} \star_M \hat{S}_{1:k,1:k,i} \star_M \hat{V}_{:,1:k,i}^T\|_F^2 \leq \|\hat{A}_{:,i} - \hat{B}_{:,i}\|_F^2$$

The result follows when we sum over indices $i = 1, \dots, n$ and we can conclude:

$$\|\hat{A} - \hat{A}_k\|_F^2 \leq \|\hat{A} - \hat{B}\|_F^2 \quad \square$$

3 CUR Decomposition

3.1 Background

One way to motivate low rank approximations for matrices stems from the linear regression problem. Recall the intuition behind regression is finding the vector that minimizes distance between some target vector b and the span of a matrix A :

$$\min_{x \in \mathbb{R}^n} \|Ax - b\|_2^2$$

The solution to this classical problem gives us the pseudoinverse of A times the target vector b :

$$x_{opt} = (A^T A)^{-1} A^T b$$

In other words, the best solution is the orthogonal projection of the target vector into the subspace spanned by A . In the space of matrices, we can apply the same intuition. We can construct a system of minimization problems relating the rows of A to some target matrix B .

$$\min_{X \in \mathbb{R}^{n \times n}} \|AX - B\|_F^2$$

In low rank approximations of matrices, we treat the matrix B as the original matrix A with the additional constraint that X must be less than some rank (we will use k for simplicity and ease of notation). We find out that the optimal solution to X is the projection matrix onto the subspace spanned by the top k singular vectors of A . In other words, X would be given by:

$$X_{opt} = P_{A,k} = V_{A,k} V_{A,k}^T$$

In essence, we have just introduced the intuition behind the Eckart-Young Theorem which tells us that the best approximation for A is given by the Singular Value Decomposition (SVD). In practice, the SVD is quite expensive to compute and in some applications, quite difficult to interpret. These struggles motivate the development of Randomized Numerical Linear Algebra.

Randomized Numerical Linear Algebra is a field that seeks to improve upon the runtime of traditional linear algebra algorithms through randomization. RandNLA takes advantage of the tradeoff of determinism for time, which says that an algorithm with some randomized elements will run faster, but will not reach an exact solution. This makes RandNLA particularly useful matrix approximation, a problem that intrinsically involves some loss of precision. In this paper, we hope to employ CUR matrix approximation to create a new way of decomposing tensors.

3.2 Definitions

The first definition will state the main result of CUR decompositions from Mahoney et al.

Definition (CUR Decomposition):

Let $A \in \mathbb{R}^{n \times n}$ be some data matrix. $C \in \mathbb{R}^{m \times c}$ is a matrix consisting of c columns of A and $R \in \mathbb{R}^{r \times n}$ is a matrix consisting of r rows of A . Then $A' = CUR$ is a column-row based matrix approximation for A or CUR matrix decomposition for any $c \times r$ matrix U . [1]

Assume for the next results that c columns of A have already been chosen from c *i.i.d* trials by randomly sampling from some probability distribution given by $\{p_i\}_{i=1}^n$ and let

i_t denote the column selected from the t -th trial (for trials $t = 1, \dots, c$).

Definition (S_C):

Let $S_C \in \mathbb{R}^{n \times c}$ be a zero-one sampling matrix where $S_{it} = 1$ and $S_{ij} = 0$

Definition (D_C):

Let $D_C \in \mathbb{R}^{c \times c}$ be a diagonal rescaling matrix where $D_{tt} = \frac{1}{\sqrt{cp_{i_t}}}$

Based on the above definitions, we can then construct our matrix $C \in \mathbb{R}^{m \times c}$ where $C = AS_C D_C$ Likewise, we can construct $R \in \mathbb{R}^{r \times n}$ using a similar procedure except now $R = D_R S_R^T A$

3.3 Subspace Sampling

One of the most important parts of CUR decomposition is the way C and R are constructed through sampling from the original matrix A . Subspace sampling is the theory that there is some underlying distribution of the data based on the Euclidean norms of the rows of the singular vectors from A . The intuition is that the right singular vectors of A contain subspace information about the span of the columns of A and the left singular vectors of A contain subspace information about the span of the rows of A [1]. By carefully selecting a subset of columns of A using the re-scaled Euclidean norms of the top k right singular vectors, the goal is to decode which columns act as a dictionary for the subspace spanned by those k singular vectors.

3.4 Algorithm

We will introduce 2 algorithms for CUR decomposition. The first randomly selects the columns/rows of the data matrix A in order to construct either C or R , and the second constructs the CUR low rank approximation.

Algorithm (ColumnSelect):

Input: rank k , failure rate δ , error term ϵ [1]

1. Compute top k right singular vectors of A given in matrix form by: $V_{A,k}$
2. Compute sampling probabilities $\{p_i\}_{i=1}^n$ where $p_i = \frac{1}{k} |(V_{A,k}^T)^{(i)}|^2$
3. For $t = 1, \dots, c$ pick $i_t \in [n]$ where $P(i_t = i) = p_i$ (recall $c = O(k^2 \log(\frac{1}{\delta}) \frac{1}{\epsilon^2}))$
4. Construct sampling matrix S and rescaling matrix D where $S_{it} = 1$ and $D_{tt} = \frac{1}{\sqrt{cp_{i_t}}}$
5. Return $C = AS_C D_C$

Algorithm (RowSelect):

Input: number of columns selected c , failure rate δ , error term ϵ

1. Compute top c left singular vectors of A given by: U_C
2. Compute sampling probabilities $\{p_i\}_{i=1}^m$ where $p_i = \frac{1}{c} |(U_C^T)^{(i)}|^2$
3. For $t = 1, \dots, r$ pick $i_t \in [m]$ where $P(i_t = i) = p_i$ (recall $r = O(c^2 \log(\frac{1}{\delta}) \frac{1}{\epsilon^2}))$

-
4. Construct sampling matrix S and rescaling matrix D where $S_{it} = 1$ and $D_{tt} = \frac{1}{\sqrt{cp_{it}}}$
 5. Return $R = D_T S_T^T A$

Algorithm (CUR Decomposition):

Input: rank parameter k , failure rate δ , error term ϵ [1]

1. Run ColumnSelect. This will give us C
2. Run RowSelect. This will give us R
3. Construct $W = D_R S_R^T C$ and compute $U = W^+$
4. Return CUR

3.5 Error Bounds

Since CUR is a matrix decomposition method based on random sampling from the original matrix, there is always a probability of failure. The following two statements put probabilistic bounds on the error of CUR decomposition.

Lemma 1. *Let $A \in \mathbb{R}^{m \times n}$ and $k \ll \min m, n$. then there exists a randomized algorithm such that exactly $c = O(k^2 \log(\frac{1}{\delta}) \frac{1}{\epsilon^2})$ columns of A are selected to construct C . Then with probability at least $1 - \delta$,*

$$\|A - CC^+A\|_F \leq (1 + \epsilon)\|A - A_k\|_F$$

[1]

Proof. Recall that $C = AS_C D_C$. Then $C^+ = (AS_C D_C)^+$ is the Moore-Penrose pseudoinverse of C . We define $P_{A,k} = U_{A,k} U_{A,k}^T$ to be the projection onto the space spanned by the top k left singular vectors of A . Then notice that: $P_{A,k} A = U_{A,k} U_{A,k}^T U_A \Sigma_A V_A^T = U_{A,k} \Sigma_A V_A^T = A_k$. Then:

$$\begin{aligned} \|A - CC^+A\|_F &\leq \|A - CC^+(P_{A,k}^+ P_{A,k})A\|_F \\ &= \|A - C(CP_{A,k})^+ A_k\|_F \\ &= \|A - (AS_C D_C)(P_{A,k} AS_C D_C)^+ A_k\|_F \\ &= \|A - AS_C D_C D_C^+ S_C^+ A_k^+ A_k\|_F \\ &\leq (1 + \epsilon)\|A - AA_k^+ A_k\|_F = (1 + \epsilon)\|A - A_k\|_F \end{aligned}$$

Note: We refer the reader to [1] for the specifics of the last line equating $AA_k^+ A_k$ and A_k . $\square$

Theorem 2. *Let $A \in \mathbb{R}^{m \times n}$ and $k \ll \min m, n$ then there exists a randomized algorithm such that exactly $c = O(k^2 \log(\frac{1}{\delta}) \frac{1}{\epsilon^2})$ columns of A are selected to construct C and exactly $r = O(c^2 \log(\frac{1}{\delta}) \frac{1}{\epsilon^2})$ rows of A are selected to construct R . Then with probability at least $1 - \delta$,*

$$\|A - CUR\|_F \leq (1 + \epsilon)\|A - A_k\|_F$$

Where U is the Moore-Penrose pseudoinverse of C and R given by: $U = (D_R S_R^T C)^+ [1]$

Proof.

$$\begin{aligned}
\|A - CUR\|_F &= \|A - C(D_R S_R^T C)^+ D_R S_R^T A\|_F \\
&= \|A - CC^+ S_R^{T+} D_R^+ D_R S_R^T A\|_F \\
&\leq (1 + \epsilon) \|A - CC^+ A\|_F \leq (1 + \epsilon)^2 \|A - A_k\|_F \\
&= (1 + 2\epsilon + \epsilon^2) \|A - A_k\|_F = (1 + \epsilon') \|A - A_k\|_F
\end{aligned}$$

□

3.6 Applications

An application of CUR decompositions is seen in data analysis work concerning DNA data of patients in a series of clinical trials. Consider data in the form of a matrix $A \in \mathbb{R}^{m \times n}$ with m being the individuals in a study and n being the genes being considered. The matrix C would be a collection of genes and the matrix R would be a collection of individuals from the study where the CUR matrix collection of genes and individuals best describe the remaining genes and individuals from the total population. [1] Another application of CUR decompositions is when the SVD is simply too large and expensive to compute. The improved time complexity of CUR decompositions makes low rank approximations more feasible as the dimensions of A increases.

4 T-CURM

We propose a new way to carry out tensor decompositions. Unlike T-SVDM, we will instead compute the CUR decomposition on each slice of the tensor to do the low rank approximation. We will call this method CUR T-SVDM Decomposition. Like CUR, the proposed CUR T-SVDM will have lower theoretical runtime than the original T-SVDM.

4.1 Algorithm

By combining the T-SVDM algorithm with a CUR decomposition, we hope to improve upon the asymptotic runtime of the algorithm, which is hurt by the cost of performing multiple calculations of a matrix's eigenvalues and vectors. We define the following algorithm.

Algorithm (T-CURM):

Input: $\mathbf{A} \in \mathbb{C}^{m \times p \times n}, \mathbf{M} \in \mathbb{C}^{n \times n}, k \in \mathbb{N}$

1. $\hat{\mathbf{A}} \leftarrow \mathbf{A} \times_3 \mathbf{M}$
2. **for** $i = 1, \dots, n$ **do**:
3. $\hat{\mathbf{C}}_{:, :, i}, \hat{\mathbf{U}}_{:, :, i}, \hat{\mathbf{R}}_{:, :, i} = CUR(\hat{\mathbf{A}}_{:, :, i}, k)$
 end for
4. $\hat{\mathbf{C}}_k = \text{stack}(\hat{\mathbf{C}}_{:, :, i})$ **for** $i = 1 \dots n$
 $\hat{\mathbf{U}}_k = \text{stack}(\hat{\mathbf{U}}_{:, :, i})$ **for** $i = 1 \dots n$
 $\hat{\mathbf{R}}_k = \text{stack}(\hat{\mathbf{R}}_{:, :, i})$ **for** $i = 1 \dots n$

5. **return** $\hat{\mathbf{C}}_k, \hat{\mathbf{U}}_k, \hat{\mathbf{R}}_k, \mathbf{M}$

Note that $\mathbf{A}$ is the tensor we are trying to decompose, $\mathbf{M}$ is an invertible matrix, and k is the rank of our decomposition.

Also note that the T-SVDM algorithm from [4] does not return a compressed version of the original tensor. The compression is left to a secondary operation that performs the $\star_M$ operator on the first k columns or rows of $\hat{\mathbf{U}}, \hat{\mathbf{S}}, \hat{\mathbf{V}}$. In T-CURM, however, this rank reduction is done within the algorithm, as the CUR algorithm requires a rank be given to calculate C, U , and R .

T-CURM returns the matrices $\hat{\mathbf{C}}_k, \hat{\mathbf{U}}_k$, and $\hat{\mathbf{R}}_k$. To reconstruct the transformed matrix $\hat{\mathbf{A}}_k$, we use the $\star_M$ operator. If we want to return the matrix to the original domain, we simply use $\mathbf{M}^{-1}$, applied to $\hat{\mathbf{A}}_k$ using $\times_3$.

$$\begin{aligned}\hat{\mathbf{A}}_k &= \hat{\mathbf{C}}_k \star_M \hat{\mathbf{U}}_k \star_M \hat{\mathbf{R}}_k. \\ \mathbf{A}_k &= \hat{\mathbf{A}}_k \times_3 \mathbf{M}^{-1}\end{aligned}$$

4.2 Results

Included below are the results of one round of testing. We input a .jpeg file, and compressed it so that the C, U, R matrices were of a given rank. We then reconstruct the original matrix using the methods described above.



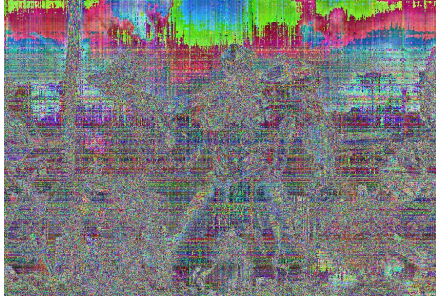
Figure 3: Original Image



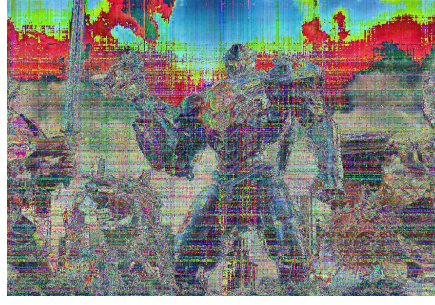
(a) Image Reconstructed from rank 100 appx.



(b) Image Reconstructed from rank 200 appx.



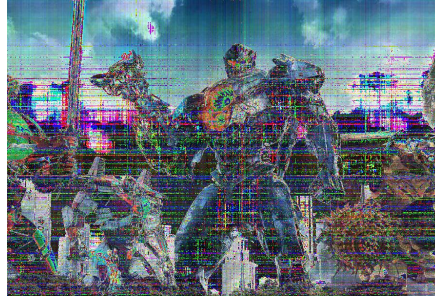
(a) Image Reconstructed from rank 300 appx.



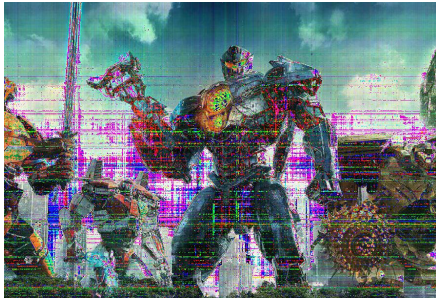
(b) Image Reconstructed from rank 400 appx.



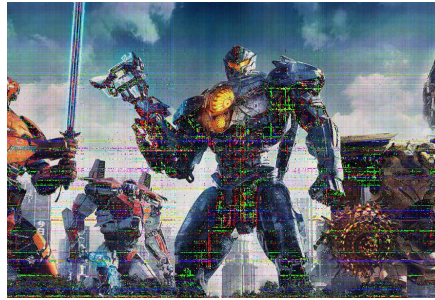
(a) Image Reconstructed from rank 500 appx.



(b) Image Reconstructed from rank 600 appx.



(a) Image Reconstructed from rank 700 appx.



(b) Image Reconstructed from rank 800 appx.

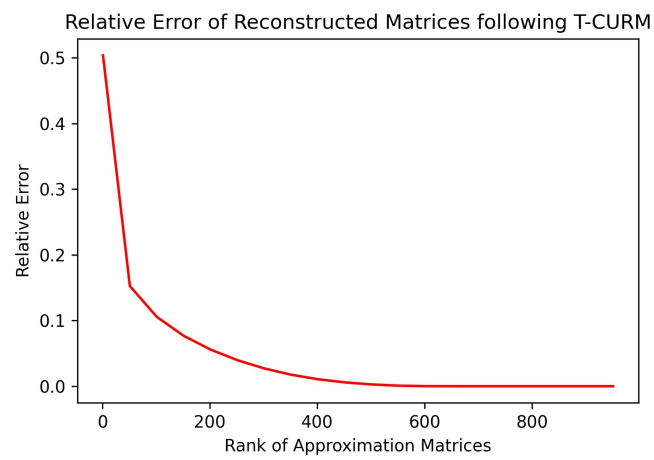


Figure 8: Relative Error for T-CURM algorithm

5 Conclusions and Future Work

Overall, we are satisfied with the performance of the T-CURM algorithm, particularly in how its relative error improves with higher rank approximations in a way that mirrors other decompositions like SVD or PCA. That said, the implementation has several issues, particularly with how it compresses image data. The CUR implementation that we use had bugs that we were able to figure out, namely the pseudo inverse matrix it chose, but using pre-implemented code definitely led to a few blind spots that a from-scratch CUR implementation would spot. We are still using $\mathbf{M} = \mathbf{I}$, as we were having trouble getting improved compression for other orthogonal $\mathbf{M}$. We can expect leaps in performance once we figure this out.

There are a number of directions that this research could go into:

1. One question which is still an open area of research for the T-SVDM and T-SVDMII is the choice of optimal orthogonal $\mathbf{M}$. Using the different lens of doing the CUR decomposition instead of the SVD, we could explore what choice in $\mathbf{M}$ leads to the most compressible tensor.
2. Throughout the project's implementation, our choice of $\mathbf{M}$ was the identity matrix. This was primarily because other options (Discrete Fourier Transform, Discrete Cosine Transform) yielded far poorer results when we tried to reconstruct images. While the identity matrix gave solid results, we would want to tackle the open question of how to choose the $\mathbf{M}$ that best reduces error for image compression.
3. Another question is to not only randomize the singular value decomposition that happens on each slice during the T-SVDM, but to build some sort of selection algorithm for the sheets themselves. This could be something along the lines of having the probability that a sheet gets selected (and then decomposed) is proportional to its nuclear norm.
4. One of the big motivators for the CUR decomposition was the increase interpretability of the outputs of the algorithm [6]. Doing the majority of our algorithm in the transform domain makes the outputs less immediately interpretable. However, by picking an orthogonal $\mathbf{M}$ in a smart way, we might still be able to learn things about the underling structure of the data though this algorithm.
5. Finally we could expand our discussion of probabilistic bounds to the spacial domain.

6 Division of Labor

We all did a little bit of everything, but these were our areas of focus:

Luca: Primary T-SVDM researcher, came up with the original idea.

Luke: Implemented the T-CURM algorithm, edited Latex document.

Josh: Primary CUR researcher, looked into probalistic bounds.

Acknowledgments

This section is optional. Here, acknowledge any assistance or contributions from advisors or institutions.

References

- [1] Petros Drineas, Michael W. Mahoney, and S. Muthukrishnan. Relative-error cur matrix decompositions, 2007.
- [2] Eric Kernfeld, Misha Kilmer, and Shuchin Aeron. Tensor–tensor products with invertible linear transforms. *Linear Algebra and its Applications*, 485:545–570, 2015.
- [3] Misha E. Kilmer, Karen Braman, Ning Hao, and Randy C. Hoover. Third-order tensors as operators on matrices: A theoretical and computational framework with applications in imaging. *SIAM Journal on Matrix Analysis and Applications*, 34(1):148–172, 2013.
- [4] Misha E Kilmer, Lior Horesh, Haim Avron, and Elizabeth Newman. Tensor-tensor algebra for optimal representation and compression of multiway data. *Proceedings of the National Academy of Sciences*, 118(28):e2015851118, 2021.
- [5] Tamara G. Kolda and Brett W. Bader. Tensor decompositions and applications. *SIAM Review*, 51(3):455–500, 2009.
- [6] Michael W Mahoney and Petros Drineas. Cur matrix decompositions for improved data analysis. *Proceedings of the National Academy of Sciences*, 106(3):697–702, 2009.
- [7] Jiani Zhang, Arvind K Saibaba, Misha E Kilmer, and Shuchin Aeron. A randomized tensor singular value decomposition based on the t-product. *Numerical Linear Algebra with Applications*, 25(5):e2179, 2018.